Actividad 1: BodyMetrics, calculadora de IMC

BodyMetrics es una app en Flutter que calcula el Índice de Masa Corporal. La idea de la actividad era cuidar la experiencia de usuario: que el formulario sea cómodo, que los controles se sientan modernos y que el resultado se entienda a la primera, con colores, una ilustración, recomendaciones y un video de ejercicio.

Repositorio en GitHub: https://github.com/esteban-lt/body_metrics

El flujo es corto: llenas tus datos, presionas Calcular, ves una pequeña animación de carga y llegas a la pantalla de resultados con una transición suave. Desde ahí puedes abrir la rutina en video o volver a calcular. Además hice los dos retos extra de modo oscuro e idioma español/inglés. Todas las capturas son del emulador de Android.

Cómo está organizado el código

Nada vive en `main.dart`; cada cosa tiene su carpeta:

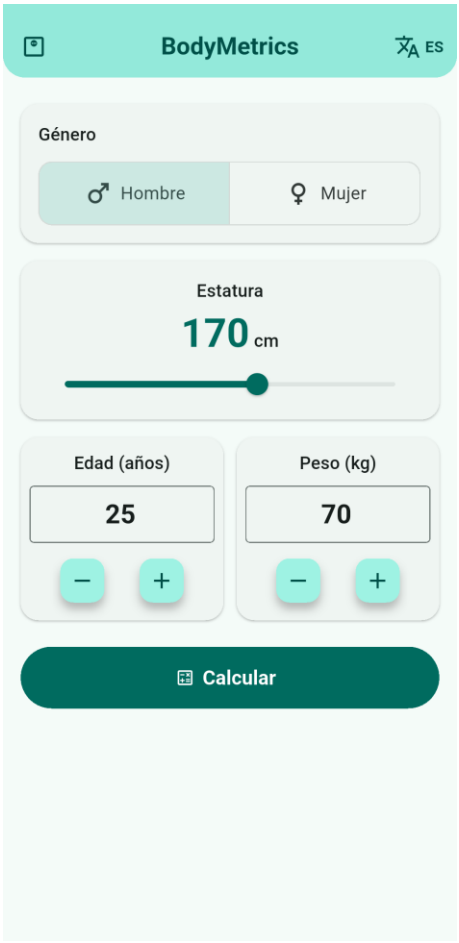
```
lib/
├─ main.dart                 MaterialApp: temas, idioma y pantalla inicial
├─ models/person_data.dart   datos del formulario (género, estatura, peso, edad)
├─ logic/bmi_calculator.dart fórmula del IMC y categorías
├─ screens/                  home_screen, result_screen, video_screen
├─ widgets/                  counter_field, gender_selector, bmi_illustration,
│                            language_button, fade_slide_route
├─ theme/app_theme.dart      tema claro/oscuro y colores de categoría
└─ l10n/app_strings.dart     textos en español e inglés
```

1. Formulario y validación en tiempo real

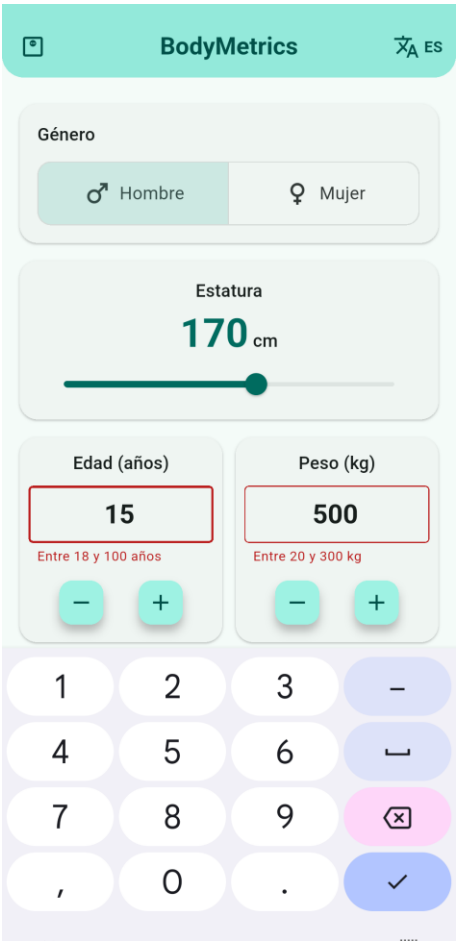
La pantalla principal usa un `Scaffold` con un `AppBar` personalizado (color de la marca, título centrado y esquinas inferiores redondeadas). Cada sección va en su propia `Card` con buen `Padding`, para que no se vea todo amontonado.

Edad y peso son `TextFormField`. Aquí hay dos capas de validación que trabajan juntas:

- **inputFormatters:** filtran lo que se escribe. En el peso solo pasan dígitos y un punto decimal; en la edad, solo dígitos. Si intentas escribir una letra, simplemente no aparece (en la prueba escribí "5abc00" y el campo se quedó con "500").
- **validator + AutovalidateMode.onUserInteraction:** revisa el rango mientras escribes: peso de 20 a 300 kg y edad de 18 a 100 años (la fórmula del IMC es para adultos). El error aparece en rojo debajo del campo sin tener que presionar nada.



Formulario al abrir la app



Peso 500 y edad 15: errores al momento, con teclado numérico

lib/widgets/counter_field.dart (el campo numérico)

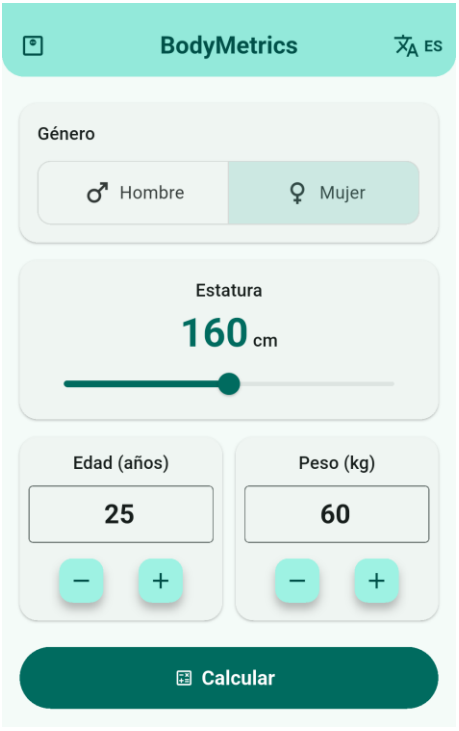
```
            TextFormField(
              controller: controller,
              textAlign: TextAlign.center,
              style: text.headlineSmall?.copyWith(fontWeight: FontWeight.bold),
              keyboardType: TextInputType.numberWithOptions(decimal: decimal),
              inputFormatters: [
                decimal
                    ? FilteringTextInputFormatter.allow(RegExp(r'^\d{0,3}(\.\d?)?'))
                    : FilteringTextInputFormatter.digitsOnly,
                LengthLimitingTextInputFormatter(decimal ? 5 : 3),
              ],
              decoration: const InputDecoration(
                errorMaxLines: 2,
                contentPadding: EdgeInsets.symmetric(horizontal: 4, vertical: 10),
              ),
              validator: validator,
```

2. Controles: slider, género y botones − / +

- **Estatura:** un `Slider` de 100 a 220 cm. Mientras lo arrastras sale una etiqueta con el valor y arriba se ve el número en grande.
- **Género:** `ToggleButtons` con iconos de hombre y mujer. Cada botón mide la mitad del ancho disponible (con `LayoutBuilder`), así que no se desborda en pantallas chicas.
- **Edad y peso:** botones `FloatingActionButton.small` de − y +. La edad sube de 1 en 1 y el peso de 0.5 en 0.5, siempre dentro del rango permitido. También se puede escribir directo en el campo.



Arrastrando el slider (etiqueta 162 cm), con Mujer elegido



Después: 160 cm, edad 25 (+1) y peso 60 (+0.5 dos veces)

3. Cálculo y categorías

El cálculo está separado de la interfaz, en `logic/bmi_calculator.dart`. La fórmula es IMC = peso (kg) / estatura (m)², y las categorías son las de la OMS para adultos:

IMC	Categoría	Color	Ilustración
menos de 18.5	Bajo peso	Azul	Figura delgada, flecha ↑
18.5 a 24.9	Peso normal	Verde	Figura media, palomita ✓
25 a 29.9	Sobrepeso	Amarillo	Figura más ancha, flecha ↓
30 o más	Obesidad	Rojo	Figura más ancha aún, flecha ↓

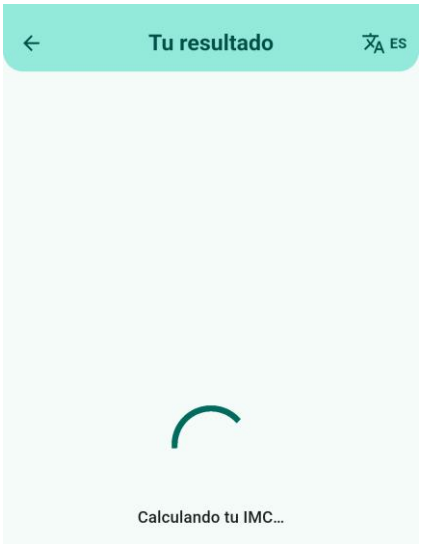
lib/logic/bmi_calculator.dart

```
/// IMC = peso (kg) / estatura (m)²
static double calculate(double weightKg, double heightCm) {
  final meters = heightCm / 100;
  return weightKg / (meters * meters);
}

/// < 18.5 bajo peso · 18.5–24.9 normal · 25–29.9 sobrepeso · ≥ 30 obesidad
static BmiCategory categorize(double bmi) {
  if (bmi < 18.5) return BmiCategory.underweight;
  if (bmi < 25) return BmiCategory.normal;
  if (bmi < 30) return BmiCategory.overweight;
  return BmiCategory.obese;
}
```

4. Animaciones: transición y carga

Al presionar Calcular, la pantalla de resultados entra con un `PageRouteBuilder` que combina un fundido (`FadeTransition`) con un deslizamiento corto desde abajo (`SlideTransition`). Ya adentro, primero se muestra un `CircularProgressIndicator` con el texto "Calculando tu IMC…" durante 1.5 segundos, y después el resultado aparece con otro fundido gracias a `AnimatedSwitcher`. Así el cambio no se siente brusco.



Animación de carga antes del resultado

lib/widgets/fade_slide_route.dart

```
Route<T> fadeSlideRoute<T>(Widget page) {
  return PageRouteBuilder<T>(
    transitionDuration: const Duration(milliseconds: 450),
    reverseTransitionDuration: const Duration(milliseconds: 300),
    pageBuilder: (_, _, _) => page,
    transitionsBuilder: (_, animation, _, child) {
      final curved =
          CurvedAnimation(parent: animation, curve: Curves.easeOutCubic);
      return FadeTransition(
        opacity: curved,
        child: SlideTransition(
          position: Tween(begin: const Offset(0, 0.08), end: Offset.zero)
              .animate(curved),
          child: child,
        ),
      );
    },
  );
}
```

5. Resultado: colores, ilustración y recomendaciones

La tarjeta principal toma el color de la categoría. El texto de encima se elige solo en negro o blanco según qué contraste mejor (`ThemeData.estimateBrightnessForColor`); por eso en el amarillo el texto es negro y se lee bien.

La ilustración es un SVG que hice yo, dibujado en el mismo código y pintado con `flutter_svg`: una figura sencilla cuyo cuerpo se ensancha según la categoría, con una insignia que indica hacia dónde conviene ir (subir, mantener o bajar). No usé imágenes de internet, así que no hay problemas de derechos de autor.

Debajo vienen tres recomendaciones que cambian con la categoría, y un resumen con los datos que capturaste. Las cuatro capturas son con la misma persona (mujer, 25 años, 160 cm), cambiando solo el peso:



60 kg → 23.4, verde



70 kg → 27.3, amarillo



85 kg → 33.2, rojo



45 kg → 17.6, azul

lib/theme/app_theme.dart (colores por categoría y contraste)

```
  static Color categoryColor(BmiCategory c, Brightness b) {
    final dark = b == Brightness.dark;
    switch (c) {
      case BmiCategory.underweight:
        return dark ? const Color(0xFF64B5F6) : const Color(0xFF1E88E5); // azul
      case BmiCategory.normal:
        return dark ? const Color(0xFF66BB6A) : const Color(0xFF2E7D32); // verde
      case BmiCategory.overweight:
        return dark ? const Color(0xFFFFD54F) : const Color(0xFFF9A825); // amarillo
      case BmiCategory.obese:
        return dark ? const Color(0xFFEF5350) : const Color(0xFFC62828); // rojo
    }
  }

  /// Texto blanco o negro, el que contraste mejor con [background]
  /// (por ejemplo, negro sobre amarillo).
  static Color onColor(Color background) =>
      ThemeData.estimateBrightnessForColor(background) == Brightness.dark
          ? Colors.white
          : Colors.black87;
```

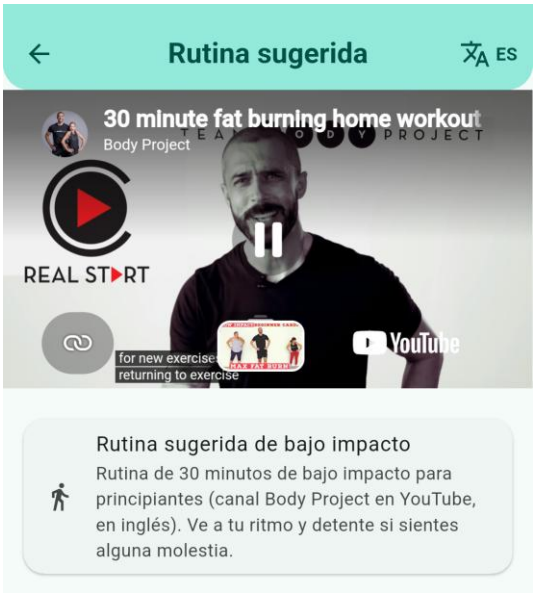
6. Video de rutina

Al final del resultado está el botón "Rutina sugerida de bajo impacto". Abre una pantalla con el reproductor de `youtube_player_flutter` y una rutina de 30 minutos para principiantes del canal Body Project. Es el mismo video para todas las categorías.

Antes de usarlo verifiqué que existe y que se puede incrustar: el endpoint oEmbed de YouTube respondió 200 para https://www.youtube.com/watch?v=gC_L9qAHVJ8. Como el título original habla de "quemar grasa", en la app le puse una etiqueta neutra para que tenga sentido también si tu resultado fue bajo peso.



El video arranca solo al abrir la pantalla

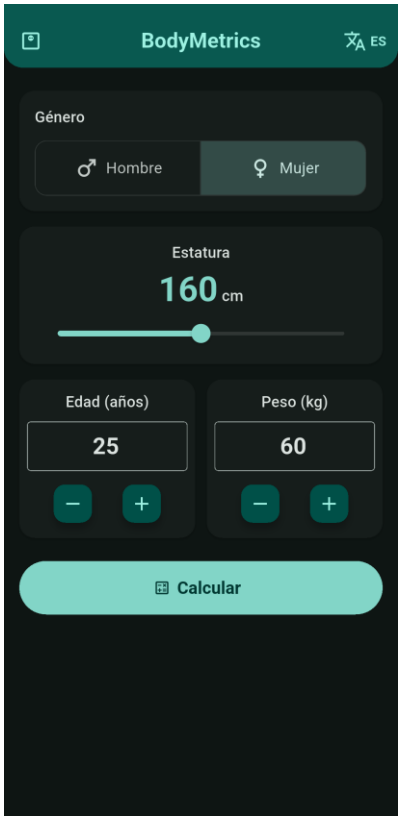


Controles del reproductor (botón de pausa: se está reproduciendo)

Un detalle técnico: la versión 10 del paquete cambió su API, ahora el controlador se crea con `YoutubePlayerController.fromVideoId(...)` y se libera con `close()` en el `dispose` de la pantalla.

7. Reto extra: modo oscuro

La app define `theme` y `darkTheme` (los dos con Material 3 a partir del mismo color semilla) y usa `ThemeMode.system`, así que cambia sola cuando cambias el modo del teléfono. Lo que más cuidé fueron los colores de categoría: en oscuro uso tonos más claros (por ejemplo, verde `#66BB6A` en vez de `#2E7D32`) y el texto de la tarjeta pasa a negro para que siga contrastando.



Formulario en modo oscuro



Resultado en modo oscuro



El mismo resultado en modo claro

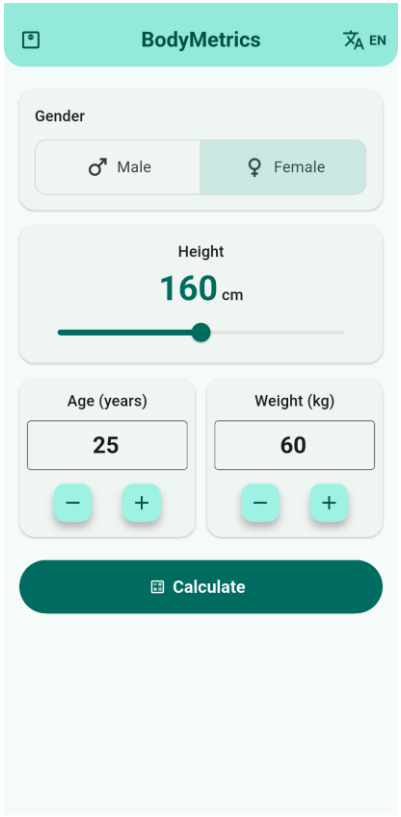
lib/main.dart

```
    return MaterialApp(
      title: 'BodyMetrics',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light(),
      darkTheme: AppTheme.dark(),
      themeMode: ThemeMode.system,
      // L10n envuelve al Navigator: todas las pantallas leen el idioma de aquí.
      builder: (context, child) => L10n(child: child!),
      home: const HomeScreen(),
```

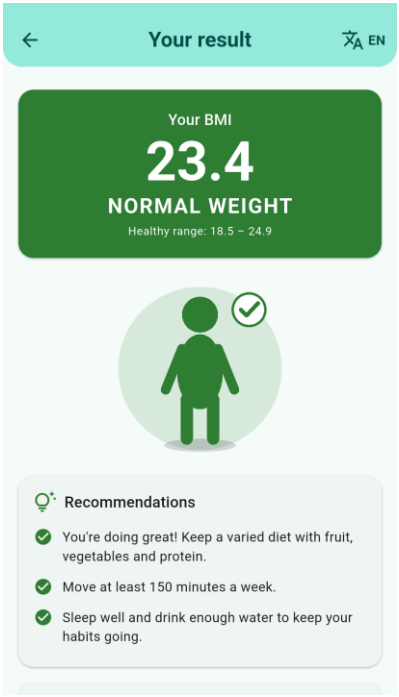
8. Reto extra: español e inglés

En la esquina del `AppBar` hay un selector de idioma (ES / EN). Al elegir uno, cambian al instante todas las etiquetas: el formulario, los mensajes de error, el resultado, las recomendaciones y la pantalla del video, sin reiniciar la app.

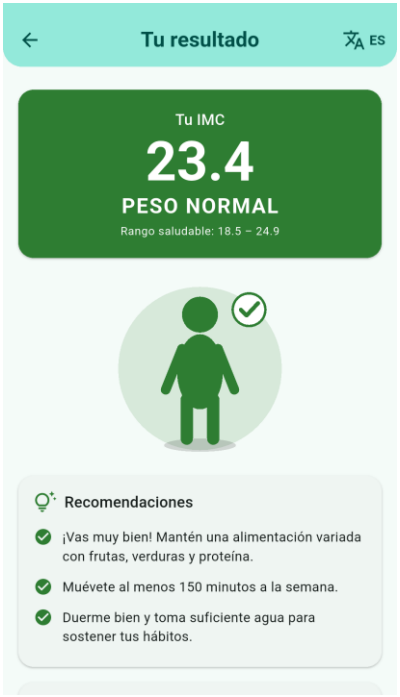
Lo hice de la forma más simple que encontré para dos idiomas: un mapa con los textos de cada idioma y un `ValueNotifier` con el idioma actual, compartido con un `InheritedNotifier`. Cada pantalla pide sus textos con `L10n.of(context)`, y eso la deja suscrita: cuando cambia el idioma, Flutter la vuelve a dibujar sola. No hizo falta `flutter_localizations` ni archivos ARB.



Formulario en inglés



Resultado y recomendaciones en inglés



El mismo resultado en español

lib/l10n/app_strings.dart

```
final appLanguage = ValueNotifier<String>('es');

/// Pone el idioma al alcance de todo el árbol. Cada widget que llama a
/// [L10n.of] queda suscrito y se redibuja cuando cambia [appLanguage].
class L10n extends InheritedNotifier<ValueNotifier<String>> {
  L10n({super.key, required super.child}) : super(notifier: appLanguage);

  static Strings of(BuildContext context) {
    final lang =
        context.dependOnInheritedWidgetOfExactType<L10n>()!.notifier!.value;
    return Strings(lang);
  }
}
```

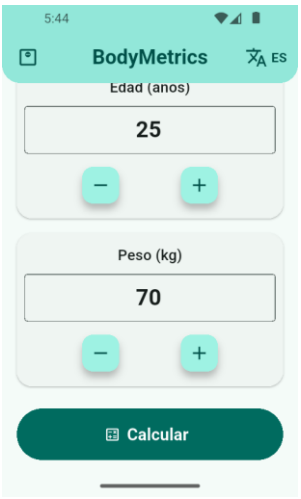
9. Pantalla pequeña frente a normal

Para probar que el diseño no se rompe, cambié la pantalla del emulador a 480×800 px con densidad 240 (`adb shell wm size 480x800` y `adb shell wm density 240`), que es como un teléfono pequeño de 320 dp de ancho. Lo que hace que se adapte:

- Todo el contenido va dentro de un `SingleChildScrollView`, así que si no cabe a lo alto, se desplaza en lugar de desbordarse.
- Edad y peso van lado a lado si hay al menos 340 dp de ancho; si no, un `LayoutBuilder` los pone uno debajo del otro.
- En pantallas grandes (tablet u horizontal) un `ConstrainedBox` limita el ancho a 600 dp para que el formulario no se estire de más.
- En el resultado, la ilustración cambia de tamaño según el ancho disponible.



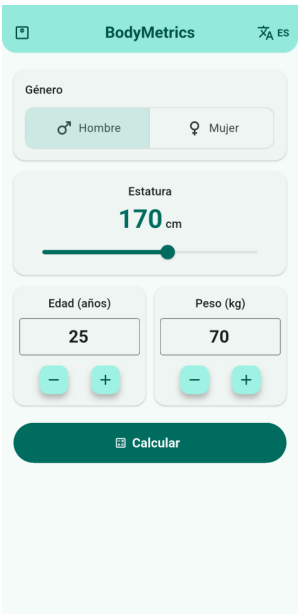
Pequeña: inicio del formulario



Pequeña: edad y peso apilados, se llega con scroll



Pequeña: resultado



Normal (1080×2400): edad y peso lado a lado